

Continual μ -Training on Edge Devices: A Survey of On-Device Personalisation

Marty Lauterbach
Reutlingen University
Reutlingen, Germany
✉ 0009-0007-3396-872X

Abstract—Wearable health devices and IoT sensors collect physiological data that varies between individuals and drifts over time. Pre-trained models optimised for population-level distributions miss this variability, while sending raw data or training gradients to the cloud raises documented privacy risks including Deep Leakage from Gradients (DLG) attacks. This paper surveys continual on-device μ -training, in which a small subset of a neural network’s parameters is updated directly on resource-constrained edge hardware. Recent work on μ -trainers, metaplastic binary networks, and label-free drift detection is organised along the four evaluation axes of Huang et al. [1]: calculation complexity, privacy, accuracy, and runtime/energy. The survey identifies that direct per-update energy measurement on wearable hardware is missing from every surveyed μ -training source, and proposes three concrete benchmark targets: per-update energy in μ J, backward-transfer accuracy on a fixed shift sequence, and drift-detection latency in samples. A first reproducible reference data point on Raspberry Pi-class hardware is reported alongside, yielding a measured $4.0\times$ marginal-energy advantage of μ -training over full-network training on the same device.

Index Terms—Wearables, μ -Training, On-Device Learning, Edge AI, Drift Detection, TinyML, Catastrophic Forgetting

I. INTRODUCTION

Wearable health devices and IoT sensors generate data shaped by individual physiological baselines, environmental exposure, and behavioural patterns. These distributions are also non-stationary: they drift as sensor hardware ages, user habits evolve, and biological conditions change. Pre-trained models, regardless of parameter count, miss this individual-level variability when deployed in personalised contexts; a model trained on population-level electrocardiogram (ECG) data degrades on a specific user’s morphology, especially for rare events. Continuous personalisation at deployment is therefore necessary.

Sending physiological data to the cloud raises privacy concerns. Even when only weight updates are transmitted, as in standard federated learning [19], those updates remain vulnerable to Deep Leakage from Gradients (DLG) attacks [6] and subsequent gradient-inversion variants [20], in which an adversary reconstructs training samples from intercepted update matrices. On-device adaptation localises the entire learning process within the device boundary and eliminates this attack

surface, but it must operate inside microcontroller memory and energy budgets measured in kilobytes of static RAM and microjoules per operation. Foundational TinyML work has progressively moved from CNN inference on commodity microcontrollers [16], through activation-memory-aware on-device transfer [17], to full backpropagation training within tight SRAM budgets [18]; the μ -training line surveyed here continues this trajectory for personalisation rather than from-scratch training.

A. Contribution

This paper makes two contributions. First, it synthesises recent work on on-device μ -trainers, metaplastic binary networks, and label-free drift detection along the four evaluation axes of Huang et al. [1]; we extend their three-axis framing (accuracy, energy, resource) by elevating privacy to a fourth axis and adding non-medical methods (UDDA-TC [4], metaplastic BNNs [7]). Second, it proposes three benchmark targets for cross-study comparison of on-device μ -training methods: (i) per-update energy in μ J on a named ARM Cortex-class platform, (ii) catastrophic-forgetting resistance measured as backward-transfer accuracy (BWT, the average change in accuracy on previously seen tasks after a new on-device update; negative values indicate forgetting) on a fixed sequence of distribution shifts, and (iii) drift-detection latency in samples between shift onset and the detector’s first positive. Federated personalisation [3] is a complementary direction not surveyed here in detail. The μ -training literature is currently dominated by a single research group [1], [2], [9]; independent reproductions remain open.

II. EVALUATION FRAMEWORK

We adopt a four-axis extension of the framing in [1]: **calculation complexity**, **privacy**, **accuracy and generalisation**, and **runtime/energy**. Privacy is elevated to a first-class axis because gradient-egress vulnerability [6] is a core motivation for on-device adaptation; runtime and energy are kept on a single axis because every primary source’s energy figure is reported on a specific hardware platform and is therefore inseparable from runtime.

Calculation complexity covers memory for weights, intermediate layer activations, and the gradients computed during training. Standard architectures need hundreds of megabytes

LaTeX source and the Raspberry Pi 5 pilot-measurement script referenced in Section VII are available at <https://github.com/mklemmingen/ContinualEdgeDevicePersonalisation>.

for one forward pass, ruling out on-device training on commodity microcontrollers without aggressive quantisation (storing each weight in fewer bits than the standard 32-bit float to shrink memory and compute); a 6-bit quantised compact CNN deployed on dedicated low-power hardware [8] achieves $0.350 \mu\text{J}$ per inference at 0.140 ms latency.

Privacy captures whether raw data, intermediate activations, or training updates leave the device. Updates alone remain vulnerable to gradient-inversion [6], [20].

Runtime and energy together determine deployability. Training a model on-device costs orders of magnitude more energy than running it for inference, so on-device adaptation must be event-driven rather than continuous.

Accuracy and generalisation justify any adaptation cycle: an update is worth its energy cost only when it produces a measurable accuracy gain on the user’s distribution. Optimising one axis tends to compromise others.

III. ON-DEVICE μ -TRAINERS

The most conservative on-device adaptation strategy freezes every layer of the pre-trained network except the final classifier and updates only that layer, restricting computation to the final feature vector and class logits. This last-layer calibration works for transfer scenarios where the deployment distribution is semantically aligned with the pre-training distribution, but its limitation is that frozen feature-extraction layers cannot acquire genuinely new representations. BioTrain [13] measures the gap directly: on EEG and EOG new-subject calibration, full-network fine-tuning outperforms last-layer updates by approximately 7 percentage points of accuracy on a low-power microcontroller under a sub-50 mW power budget.

The main line of work in on-device personalisation extends this idea by selecting a small subset of *intermediate* layers as trainable while keeping the rest *frozen* (frozen layers receive no weight updates and contribute no gradients, so neither their values nor the intermediate activations needed to compute their gradients have to be stored on-device). Developed by Huang et al. [2], [9] for secure real-time processing of medical signals (continuous ECG and electroencephalogram monitoring), this μ -training approach recovers more representational flexibility than last-layer calibration without paying the full memory cost of training every layer.

A. Tripartite Pipeline

The original μ -trainer follows a three-phase pipeline.

Phase 1: Full Cloud Training. A compact model is trained on a large aggregated dataset, e.g. the Telehealth Network of Minas Gerais ECG repository [2]. The model is structured in three blocks: an input encoder, an intermediate processing layer, and an output decoder.

Phase 2: μ -Training (pre-deployment). Encoder and decoder are initialised from the trained foundational model and frozen; only the middle layer is updated. The original middle-layer training used a separate, larger teacher network as a supervision target [2]; the follow-up work [9] eliminates the on-device teacher by using the model as its own teacher (*self-distillation*).

Phase 3: On-Device μ -Fine-Tuning. Once deployed (evaluated on Raspberry Pi 3 Model B [2] and Radxa Zero [9]), the freezing logic inverts: the middle layer is frozen to preserve its distilled representations while the encoder and decoder are unfrozen and updated on personal data when user-specific drift is detected.

B. Parameter Efficiency and Privacy

μ -trainers work because the on-device trainable subset is small. For the specific 6.4M-parameter CNN foundational model evaluated in [2], less than 2 % of weights are updated on-device; the absolute fraction is architecture-dependent but the order of magnitude is representative. Updates fit within on-chip memory, avoiding off-chip data transfers and remaining within microjoule energy budgets.

Localising raw data, activations, and weight updates within the device boundary eliminates the attack surface for gradient-inversion attacks [6]. Because the device does not send updates to a global aggregator, the entire DLG attack family is inapplicable by construction, giving a strong privacy guarantee for regulated health data.

IV. CATASTROPHIC FORGETTING AND METAPLASTIC BNNs

Continual on-device updates risk overwriting safety-critical features: an ECG monitor that adapts its weights to a user’s resting heart-rate variability could overwrite the parameters needed to detect rare arrhythmic events. Standard mitigations (replay buffers that rehearse stored past samples alongside new data [21], and importance-weighted regularisation that protects weights deemed important to past tasks [10]) both add memory or compute that wearable microcontrollers cannot spare.

Drawing on neuroscience, Laborieux et al. [7] propose Binary Neural Networks (BNNs) as a forgetting-resistant on-device substrate. BNNs constrain weights to ± 1 at inference time, allowing integer-only operations rather than floating-point arithmetic; training nonetheless accumulates small updates in a hidden real-valued shadow per binary weight, binarised only at the inference boundary. Laborieux et al. reinterpret that shadow’s magnitude as a per-weight stability score: weights with large hidden magnitudes become progressively resistant to flipping while small-magnitude weights remain plastic. The same architectural mechanism that gives efficient inference also gives forgetting resistance, without the memory overhead of replay or the compute overhead of [10]. The result is demonstrated on multitask and stream-learning vision benchmarks; demonstration on streaming physiological signals (ECG, EEG, IMU) remains an open empirical step.

V. LABEL-FREE DRIFT DETECTION

On-device adaptation cannot run continuously without exhausting the energy budget; updates must be triggered by statistically significant changes in the data distribution. In deployed edge systems, distributions drift continuously as user behaviour and sensor characteristics change. Classical

streaming drift detectors (DDM [22], CUSUM-based change-point methods) mostly assume access to ground-truth labels for incoming data, which is impractical on a wearable where the user cannot annotate their own physiological stream in real time. Label-free drift detection is therefore a precondition for autonomous on-device adaptation.

The Unsupervised Real-Time Drift Detection and Adaptation method for continual Traffic Classification (UDDA-TC) [4] implements a two-tier detector. The first tier is a sliding-window adaptive weighted CUSUM detector: it accumulates small deviations from an expected mean and signals when the cumulative breach exceeds a threshold, indicating sustained shift rather than noise, with the sliding window bounding memory consumption. The second tier introduces a Weighted Logits Variation (WLV) signal. Logits are the model’s raw per-class prediction scores before softmax normalisation; WLV aggregates the variance of these scores over a sliding window, weighted by class prior. The resulting one-dimensional signal serves as a proxy for input drift, computable on edge hardware where direct distributional analysis of high-dimensional inputs would be prohibitive. On joint detection, the system reconstructs a training set by merging the emerging distribution with retained anchor points from the historical distribution, producing a compact batch suitable for a subsequent μ -fine-tuning cycle.

A complementary line of work places the detection target inside the network rather than on its inputs or outputs. Three published frameworks instantiate this idea along distinct signals: prototype-distance in a contrastive latent space [11], distribution distance over a sliding representation window [12], and trajectory deviation in the DIKWP semantic-transformation framework of Mei et al. [5]. These detectors could complement UDDA-TC’s output-side trigger by catching shifts that affect intermediate representations before they reach the prediction layer, but none has been validated on wearable microcontroller hardware; for that reason they are not included as separate rows in the comparative analysis of Section VI, and their adaptation to the μ -training setting has not yet been attempted.

VI. COMPARATIVE ANALYSIS

Table I summarises the surveyed methods across the four axes of Section II. Cross-row magnitudes compare design properties rather than benchmark performance (rows span biosignals, vision, and encrypted traffic); cells where the source does not report a comparable metric are qualitative.

When privacy and calculation complexity dominate, as in implanted medical devices or secure hearable biosensors, μ -training is the current state of the art: less than 2% of model parameters are updated, operations fit within on-chip memory, and update egress is eliminated [2], [9]. For workloads needing continuous real-time runtime alongside forgetting resistance, metaplastic BNNs are the architectural candidate, though their empirical case has been made on vision streams rather than on physiological time series [7]. Label-free drift detection is a necessary auxiliary regardless of the core adaptation algorithm:



Fig. 1. Pilot-measurement testbed referenced by the data point in the energy-asymmetry discussion below. A Raspberry Pi 5 hosts the μ -training, inference, and drift-detection workloads. The ATORCH J7-c USB-C inline meter visible in the foreground (reading 5.13 V, 1.07 A) is used as a sanity-check reference; the reported power figures themselves come from the Pi 5’s on-chip Renesas DA9091 PMIC, queried via `vccgencmd pmic_read_adc`. The PMIC measures the secondary rails it generates (3V3, 1V8, VDD_CORE, DDR, ...) but not the 5 V main-rail current directly, so peripheral draw (HAT, fan, and cabling visible in frame) is excluded from totals; relative comparisons across phases remain valid because the unmeasured offset is approximately constant.

adaptation is energetically viable only when triggered by genuine distributional shift [4].

VII. OPEN CHALLENGES AND QUANTITATIVE BOUNDS

Catastrophic forgetting under tight memory. Partial freezing and metaplasticity offer pathways but have not been validated end-to-end on streaming physiological signals over long horizons. Cross-modality demonstration on ECG, EEG, or IMU streams remains the open empirical step for both μ -trainers and metaplastic BNNs [7]. As a sanity check on the methodology proposed below in (ii), we ran a synthetic two-task forgetting probe on the same Pi 5 testbed, scored with the backward-transfer (BWT) metric of Lopez-Paz & Ranzato [23] (input mean shift and label rotation between tasks for the synthetic construction): full-network training collapses to $\text{BWT} = -1.0$ (complete forgetting of task 0 after training on task 1), confirming that the failure mode targeted by μ -training and metaplasticity is real and immediately reproducible on commodity hardware.

Energy asymmetry between inference and training. Inference is efficient: $0.350 \mu\text{J}$ at 0.140 ms latency on dedicated low-power hardware [8], well within the energy budget of a wearable-class battery for continuous monitoring. The corresponding per-update training energy is not directly reported in any of the surveyed μ -training sources. The wider TinyML training literature provides reference points that bracket the achievable measurement envelope: per-iteration training energy on low-end [14] and mid-range [15] microcontrollers, and full-network biosignal fine-tuning under 50 mW on the GAP9 RISC-V MCU [13].

To anchor this gap with a first reproducible reference point on the same Pi-class hardware that the surveyed μ -training works deploy on (Raspberry Pi 3 [2], Radxa Zero [9]), we

TABLE I
COMPARATIVE ANALYSIS OF ON-DEVICE ADAPTATION METHODOLOGIES

Approach	Calculation Complexity	Privacy	Accuracy & Generalisation	Runtime & Energy
μ -Training [2], [9]	Very Low (<2 % of weights updated, architecture-dependent [2])	Very High (no update egress)	Improved over static foundational model on TNMG/CPSC ECG [2]	On-chip updates on Raspberry Pi 3 [2] and Radxa Zero [9]; this work measures 33.8 ms/118 mJ per update on Pi 5 ($4.0\times$ less than full-network)
Metaplastic BNN [7]	Ultra-Low (integer-only operations)	High (binary weights obfuscate raw input)	Matches [10] on permuted-MNIST and similar multitask benchmarks [7]; no physiological evaluation reported	Efficient inference; per-update μ J figure not reported
Label-Free Drift (UDDA-TC) [4]	Low (logit variance + sliding window)	High (fully unsupervised; no label egress)	Drift-detection F1 0.71–0.76 across three traffic-classification corpora [4]	Sliding-window bounds memory; per-decision compute comparable to baseline change detectors

ran the testbed of Fig. 1 through a trainable-fraction sweep on a Raspberry Pi 5 with on-chip PMIC power telemetry (via `vcgencmd pmic_read_adc`). With the μ -training freeze pattern (2.06 % of weights trainable on a 99.7 k-parameter ECG-shaped 1D-CNN), each update costs 33.8 ms and 118 mJ above the 2.77 W Pi 5 idle baseline, versus 137.9 ms and 475 mJ for full-network training: a $4.08\times$ wall-time and $4.02\times$ marginal-energy advantage measured on the same device (Fig. 2). The Pi 5 measurement sits 3–4 orders of magnitude above the $10^2 \mu$ J/update Cortex-M target proposed below; the next step is to repeat the sweep on Cortex-M4F/M7 hardware to close the gap.

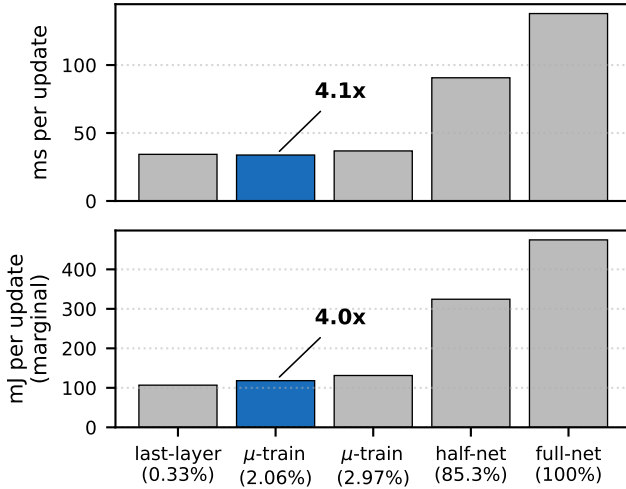


Fig. 2. Per-update wall-time (top) and marginal energy above the 2.77 W Pi 5 idle baseline (bottom) versus trainable-parameter fraction. The μ -training operating point at 2.06 % trainable (blue) sits on the flat regime; the cost climbs sharply above $\sim 3\%$ trainable as additional convolutional layers enter the backward pass. Annotations give the measured μ -training vs. full-network ratios. Raspberry Pi 5, 4 threads, batch 32, seq. length 250, FP32, SGD; PMIC-summed power excludes peripherals on the unmeasured 5 V main rail (see Fig. 1 caveat).

Label-free drift-detection calibration. The two-tier UDDA-TC detector bounds memory and compute, but its threshold parameters are tuned empirically rather than derived. UDDA-TC reports drift-detection F1 in the range 0.71–0.76 on three traffic-classification corpora [4], leaving open the question of robustness on high-noise wearable signals where distributional shift is gradual rather than abrupt. Our Pi 5 implementation of a UDDA-TC-style WLV+CUSUM detector adds 36.6 ms and 121 mJ per decision (above idle), within 3 % of the cost of a plain FP32 inference call on the same network: detector overhead is negligible at this scale, so the cost driver remains the inference itself, not the drift signal.

Toward a shared benchmark. Three concrete targets: (i) per-update energy in μ J on a named ARM Cortex-class platform (e.g. Cortex-M4F or Cortex-M7). One “update” is the parameter-update sequence triggered by a single detected drift event: typically one mini-batch over the new-data window plus retained anchor points. Optimizer state, the auxiliary memory used by the training algorithm itself such as Adam’s per-weight running averages, is held constant across compared methods. A working ceiling on the order of $10^2 \mu$ J/update is consistent with the multi-day continuous-operation budget of a 320 mAh-class wearable cell [8]. (ii) Catastrophic forgetting measured as backward-transfer accuracy on a fixed sequence of ≥ 5 distribution shifts, for example a held-out subject sequence from a public biosignal corpus such as MIT-BIH (ECG) or PhysioNet 2017, with the shift schedule and anchor-retention policy fixed across methods. (iii) Drift-detection latency, measured in samples between a labelled shift onset and the detector’s first positive, reported at a fixed operating point (the threshold yielding a 0.05 false-positive rate over a no-shift control window of fixed length, e.g. 10,000 samples) so that latency cannot be reduced by accepting more false alarms.

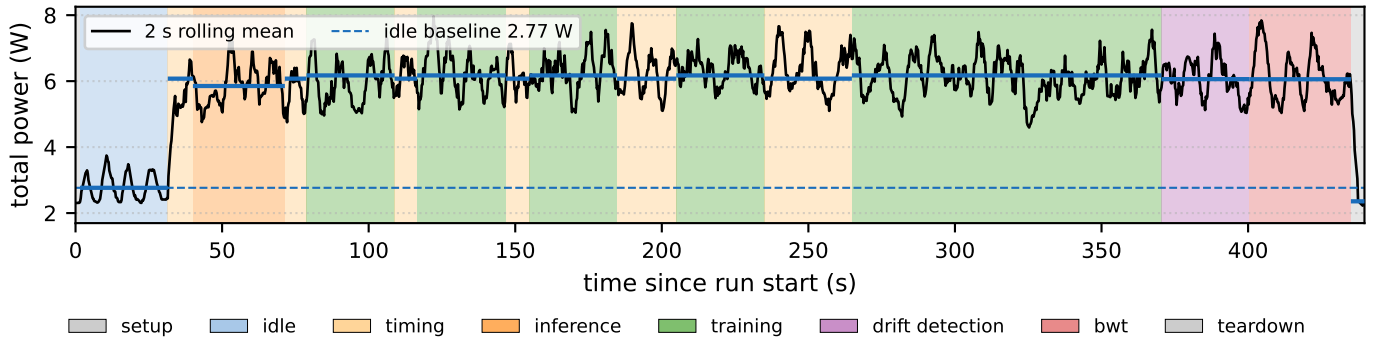


Fig. 3. Power telemetry over the full 440s pilot run. Background colour bands indicate the active pipeline phase (legend below); the thin grey trace is raw 5 Hz PMIC samples (`vcgencmd pmic_read_adc` summed across the 14 measured rails) and the thick black trace is a 2 s rolling mean. Short blue horizontal segments are per-phase mean-power overlays; the dashed line is the 2.77 W idle reference. The trainable-fraction sweep is visible as the five training plateaus near the centre, with the largest two (*half-net* and *full-net*) distinguishable by their longer settle time per update.

VIII. CONCLUSION

Inference at sub-millisecond latency and sub-microjoule per-inference cost has been demonstrated on dedicated low-power hardware [8]; the corresponding per-update training figures for the surveyed μ -training methods have not been reported. The Pi 5 reference point measured in Section VII gives a $4.0\times$ μ -training-over-full-network advantage on commodity hardware; the three benchmark targets proposed there are tractable next measurement steps that would enable direct cross-study comparison on Cortex-M-class targets.

REFERENCES

- [1] Z. Huang, L. Yu, L. F. H. Contreras, G. Dwivedi, and O. Kavehei, "On-device machine learning for secure, generalizable and real-time medical data processing: on the emergence of μ -trainers," *Royal Society Open Science*, vol. 12, no. 12, p. 251213, Dec. 2025.
- [2] Z. Huang, L. Yu, L. F. H. Contreras, K. Eshraghian, N. D. Truong, A. Nikpour, and O. Kavehei, "Advancing privacy-aware machine learning on sensitive data via edge-based continual μ -training for personalized large models," *Machine Learning: Science and Technology*, vol. 6, no. 1, p. 015025, Jan. 2025, doi: 10.1088/2632-2153/adaca3.
- [3] H. Lin, L. Shou, K. Chen, G. Chen, and S. Wu, "LINDT: Tackling negative federated learning with local adaptation," *arXiv preprint arXiv:2011.11160* [cs.LG], Nov. 2020.
- [4] M. Liu, P. Wang, Z. Li, Y. Ye, Z. Wang, and X. Chen, "UDDA-TC: Unsupervised real-time drift detection and adaptation for continual traffic classification in mobile edge computing," *IEEE Transactions on Consumer Electronics*, early access, 2025, doi: 10.1109/TCE.2025.3579882.
- [5] Y. Mei, Y. Duan, K. Wu, and Y. Li, "A DIKWP white-box semantic distributed learning approach for resource-constrained edge devices in web environments," *International Journal of Web Information Systems*, vol. 22, no. 1, pp. 17–43, 2026, doi: 10.1108/IJWIS-06-2025-0142.
- [6] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [7] A. Laborieux, M. Ernout, T. Hirtzlin, and D. Querlioz, "Synaptic metaplasticity in binarized neural networks," *Nature Communications*, vol. 12, no. 2549, May 2021, doi: 10.1038/s41467-021-22768-y.
- [8] T. Ling, C. Qian, P. Zdankin, T. Weis, and G. Schiele, "Strike-Watch: Wrist-worm gait recognition with compact time-series models on low-power FPGAs," in *Proc. 2025 IEEE Annual Congress on Artificial Intelligence of Things (AIoT)*, Dec. 2025, pp. 66–74, doi: 10.1109/aiot66900.2025.00021.
- [9] Z. Huang, L. Yu, L. F. Herbozo Contreras, and O. Kavehei, "Efficient and secure μ -Training and μ -Fine-Tuning for edge-based TinyML with future-guided self-distillation," *Mach. Learn.: Health*, vol. 1, no. 1, p. 015002, Jul. 2025, doi: 10.1088/3049-477X/add26a.
- [10] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. National Academy of Sciences (PNAS)*, vol. 114, no. 13, pp. 3521–3526, 2017, doi: 10.1073/pnas.1611835114.
- [11] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Security Symposium*, 2021, pp. 2327–2344.
- [12] S. Greco, B. Vacchetti, D. Apiletti, and T. Cerquitelli, "Unsupervised concept drift detection from deep learning representations in real-time," *IEEE Transactions on Knowledge and Data Engineering*, 2025, arXiv:2406.17813.
- [13] R. Wang, V. J. B. Jung, P. Wiese, S. Frey, G. Spacone, F. Conti, A. Burrello, and L. Benini, "BioTrain: Sub-MB, Sub-50 mW on-device fine-tuning for Edge-AI on biosignals," in *Proc. 2026 Int. Joint Conf. Neural Networks (IJCNN)*, IEEE, 2026, accepted, arXiv:2604.13359.
- [14] S. G. Patil, P. Jain, P. Dunnmon, N. Carbin, M. Rana, and J. E. Gonzalez, "POET: Training neural networks on tiny devices with integrated rematerialization and paging," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 162, 2022, pp. 17573–17583.
- [15] L. Zhu, L. Hu, J. Lin, W.-C. Wang, W.-M. Chen, C. Gan, and S. Han, "PockEngine: Sparse and efficient fine-tuning in a pocket," in *Proc. 56th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2023, pp. 1381–1396, doi: 10.1145/3613424.3614307.
- [16] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11711–11722.
- [17] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce memory, not parameters for efficient on-device learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11285–11297.
- [18] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256 KB memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 22941–22954.
- [19] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR, vol. 54, 2017, pp. 1273–1282.
- [20] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients — how easy is it to break privacy in federated learning?," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 16937–16947.
- [21] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5533–5542.
- [22] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, LNCS, vol. 3171, Springer, 2004, pp. 286–295.
- [23] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," in *NeurIPS*, vol. 30, 2017, pp. 6470–6479.